Week-1

Information Security Systems

Write a code to find the total number of prime numbers between 1 and 1000.

```
#include<bits/stdc++.h>
using namespace std;
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
int main()
{
   vector<bool> flag(1000, true);
   flag[0] = false;
   flag[1] = false;
   for(int i = 2; i * i < 1000; i++){
      if(flag[i]){
         for(int j = i * i; j < 1000; j += i){
            flag[j] = false; }}}
int cnt=0;
   for(int i = 2; i < 1000; i++){
      if(flag[i]) {cout << i << " ";cnt++;}}
  cout<<endl<<"the final count of the number of primes is:"<<cnt;
   return 0;}
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 139 149 151 157 163 167 173 179 18
1 191 193 197 199 211 223 227 229 233 239 241 251 257 263 269 271 277 281 283 293 307 311 313 317 331 337 347 349 353 359 367 373 379 383
 389 397 401 409 419 421 431 433 439 443 449 457 461 463 467 479 487 491 499 503 509 521 523 541 547 557 563 569 571 577 587 593 599 601
607 613 617 619 631 641 643 647 653 659 661 673 677 683 691 701 709 719 727 733 739 743 751 757 761 769 773 787 797 809 811 821 823 827 8
29 839 853 857 859 863 877 881 883 887 907 911 919 929 937 941 947 953 967 971 977 983 991 997
the final count of the number of primes is:168
```

Write a program to implement matrix multiplication of 3 x 3 matrix .

```
#include <iostream>
#include<bits/stdc++.h>
using namespace std;
int main()
{
   cout<<"enter number of rows"<<endl;
   int m;cin>>m;
   cout<<"enter number of columns"<<endl;
   int n;cin>>n;
   int arr[m][n];int x;
   for(int i=0;i<m;i++){
```

```
    for(int j=0;j<n;j++){
        cin>>x;
        arr[i][j]=x;}}
  cout<<"enter the number of rows for the matrix 2"<<endl;
  int m1; cin>>m1;
  cout<<"enter the number of columns for the matrix 2"<<endl;
  int n1;cin>>n1;
  int ar[m1][n1];
  for(int i=0;i<m1;i++){
    for(int j=0;j<n1;j++){
        cin>>x;
        ar[i][j]=x;}}
  if(m1!=n) return 0;
  int final[m][n1]; int s=0;
  for(int i=0;i<m;i++){
    for(int j=0;j<n1;j++){
         s=0;
        for(int k=0;k<m1;k++){
            s+=arr[i][k]*ar[k][j];}
         final[i][j]=s;   }}
  for(int i=0;i<m;i++){
    for(int j=0;j<n1;j++){
        cout<<final[i][j]<<"  ";}
    cout<<endl;}}
```

```
enter number of rows
3
enter number of columns
3
1 2 3 4 5 6 7 8 9
enter the number of rows for the matrix 2
3
enter the number of columns for the matrix 2
3
1 2 3 4 5 6 7 8 9
30  36  42
66  81  96
102  126  150
```

Write a program to find the inverse of a 3 x 3 matrix.

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;
int main() {
    double a[3][3], inv[3][3];
```

```
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        cin >> a[i][j];
double det =
    a[0][0]*(a[1][1]*a[2][2] - a[1][2]*a[2][1]) -
    a[0][1]*(a[1][0]*a[2][2] - a[1][2]*a[2][0]) +
    a[0][2]*(a[1][0]*a[2][1] - a[1][1]*a[2][0]);
if (fabs(det) < 1e-9) {
    cout << "Matrix is singular or not invertible\n";
    return 0;}
inv[0][0] =  (a[1][1]*a[2][2] - a[1][2]*a[2][1]) / det;
inv[0][1] = -(a[0][1]*a[2][2] - a[0][2]*a[2][1]) / det;
inv[0][2] =  (a[0][1]*a[1][2] - a[0][2]*a[1][1]) / det;
inv[1][0] = -(a[1][0]*a[2][2] - a[1][2]*a[2][0]) / det;
inv[1][1] =  (a[0][0]*a[2][2] - a[0][2]*a[2][0]) / det;
inv[1][2] = -(a[0][0]*a[1][2] - a[0][2]*a[1][0]) / det;
inv[2][0] =  (a[1][0]*a[2][1] - a[1][1]*a[2][0]) / det;
inv[2][1] = -(a[0][0]*a[2][1] - a[0][1]*a[2][0]) / det;
inv[2][2] =  (a[0][0]*a[1][1] - a[0][1]*a[1][0]) / det;
cout << fixed << setprecision(6);
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++)
        cout << inv[i][j] << " ";
    cout << "\n";}
return 0;}
```

```
1 2 3 3 2 1 9 7 8
-0.750000 -0.416667 0.333333
1.250000 1.583333 -0.666667
-0.250000 -0.916667 0.333333
```

Week-2

Information Security Systems

1. **Write a program to implement a substitution cipher.**

MONOALPHABETIC

CODE:

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;
int main() {
    srand(time(0));
    string plain;
    char keyU[26],
    keyL[26]; for (int i = 0; i
    < 26; i++) {
  keyU[i] = 'A' + i;
  keyL[i] = 'a' + i;}
for (int i = 0; i < 26; i++) { int
  j = rand() % 26;
  char temp = keyU[i];
  keyU[i] = keyU[j];
  keyU[j] = temp; temp =
  keyL[i]; keyL[i] =
  keyL[j]; keyL[j] = temp;}
    cout << "Enter plaintext: ";
    getline(cin, plain);
    string cipher = "";
    for (char ch : plain)
    {
      if (ch >= 'A' && ch <=
        'Z') cipher += keyU[ch
        - 'A'];
```

```
        else if (ch >= 'a' && ch <=
          'z') cipher += keyL[ch -
          'a'];
    else original+=c;}
      cout<<"original text was:"<<"
      "<<original; return 0;}
```

```
Enter plaintext: Tahira is Great
Ciphertext: Wfoqpf qy Xpdfw
original text was: Tahira is Great
```

POLYALPHABETIC

CODE:

```
  #include
  <iostream>     using
  namespace  std;  int
  main() {
      string plain, key;
      cout << "Enter plaintext: ";
      getline(cin, plain);
      cout << "Enter key:
      "; getline(cin, key);
      string cipher = "";
      string decipher = "";
      int j = 0; // key index
for (int i = 0; i < plain.length(); i++) {
   char p = plain[i];
   char k = key[j % key.length()]; if
   (p >= 'A' && p <= 'Z') {
            char c = (p - 'A' + (toupper(k) - 'A')) % 26 +
            'A'; cipher += c;
            j++;}
   else if (p >= 'a' && p <= 'z') {
            char c = (p - 'a' + (tolower(k) - 'a')) % 26 + 'a';
            cipher += c;
            j++;}
   else {
            cipher += p; }}
      cout << "Ciphertext: " << cipher <<
```

```
        endl; j = 0;
for (int i = 0; i < cipher.length(); i++) {
   char c = cipher[i];
   char k = key[j % key.length()];
    if (c >= 'A' && c <= 'Z') {
            char p = (c - 'A' - (toupper(k) - 'A') + 26) % 26 + 'A';
            decipher += p;
            j++; }
   else if (c >= 'a' && c <= 'z') {
            char p = (c - 'a' - (tolower(k) - 'a') + 26) % 26
            + 'a'; decipher += p;
      j++;}
   else {
            decipher += c;}}
       cout << "Deciphered text: " << decipher <<
       endl; return 0;}
```

```
Enter plaintext: Tahira is great
Enter key: Nisha
Ciphertext: Gizprn qk nrril
Deciphered text: Tahira is great
```

2. Write a program to implement a Frequency Analysis of given file.

```
   #include <iostream>
   #include <fstream>
   #include <cctype>
   using namespace std;
   int main() {
      ifstream file("input.txt");  // file
      name int freq[26] = {0};
      int total = 0;
      char ch;
      if (!file) {
   cout << "Error opening file!" << endl; return
   1;}
while (file.get(ch)) { if
   (isalpha(ch)) {
            ch = tolower(ch);
            freq[ch - 'a']++;
            total++;}}
```

```
    file.close();
    cout << "Character Frequency
    Analysis\n"; for (int i = 0; i < 26; i++) {
if (freq[i] > 0) {
        double percent = (freq[i] * 100.0) /
        total; cout << char('a' + i) << " : "
            << freq[i] << " ("
            << percent <<
    "%)\n";}} return 0;}
```

input.txt

```
1   tahira mehta is a good girl.
```

```
Character Frequency Analysis
a : 4 (18.1818%)
d : 1 (4.54545%)
e : 1 (4.54545%)
g : 2 (9.09091%)
h : 2 (9.09091%)
i : 3 (13.6364%)
l : 1 (4.54545%)
m : 1 (4.54545%)
o : 2 (9.09091%)
r : 2 (9.09091%)
s : 1 (4.54545%)
t : 2 (9.09091%)
```

3. Write a program to implement an affine cipher

```
#include <bits/stdc++.h>
using namespace std;
int findModInverse(int a) {
   a = a % 26;
   for (int i = 1; i < 26; i++) {
      if ((a * i) % 26 == 1)
         return i;
   }
   return -1;
}

char encryptChar(char ch, int a, int b) {
   char base = isupper(ch) ? 'A' : 'a';
   int x = ch - base;
   return char((a * x + b) % 26 + base);
}

char decryptChar(char ch, int a_inv, int b) {
   char base = isupper(ch) ? 'A' : 'a';
   int y = ch - base;
   return char((a_inv * (y - b + 26)) % 26 + base);
}

string encryptText(const string &plain, int a, int b) {
```

```
    string cipher = "";
    for (char ch : plain) {
        if (isalpha(ch))
            cipher += encryptChar(ch, a, b);
        else
            cipher += ch;
    }
    return cipher;
}

string decryptText(const string &cipher, int a, int b) {
    string plain = "";
    int a_inv = findModInverse(a);

    for (char ch : cipher) {
        if (isalpha(ch))
            plain += decryptChar(ch, a_inv, b);
        else
            plain += ch;
    }
    return plain;
}

int main() {
    string message;
    int a, b;

    cout << "Enter plaintext: ";
    getline(cin, message);

    cout << "Enter key 'a' (must be coprime with 26): ";
    cin >> a;
    cout << "Enter key 'b': ";
    cin >> b;

    if (__gcd(a, 26) != 1) {
        cout << "Error: Key 'a' is not valid!\n";
        return 0;
    }

    string encrypted = encryptText(message, a, b);
    cout << "\nEncrypted Text: " << encrypted << endl;

    string decrypted = decryptText(encrypted, a, b);
    cout << "Decrypted Text: " << decrypted << endl;

    return 0;
}
```

```
Enter plaintext: hello world
Enter key 'a' (must be coprime with 26): 5
Enter key 'b': 8

Encrypted Text: rclla oaplx
Decrypted Text: hello world
```